

Sri Lanka Institute of Information Technology

Human Computer Interaction (HCI)



IT3060 – HCI Lab Exercise 05
Activity 4: Design a High-Level Architecture Bandara A.M.K.U.
It23628540

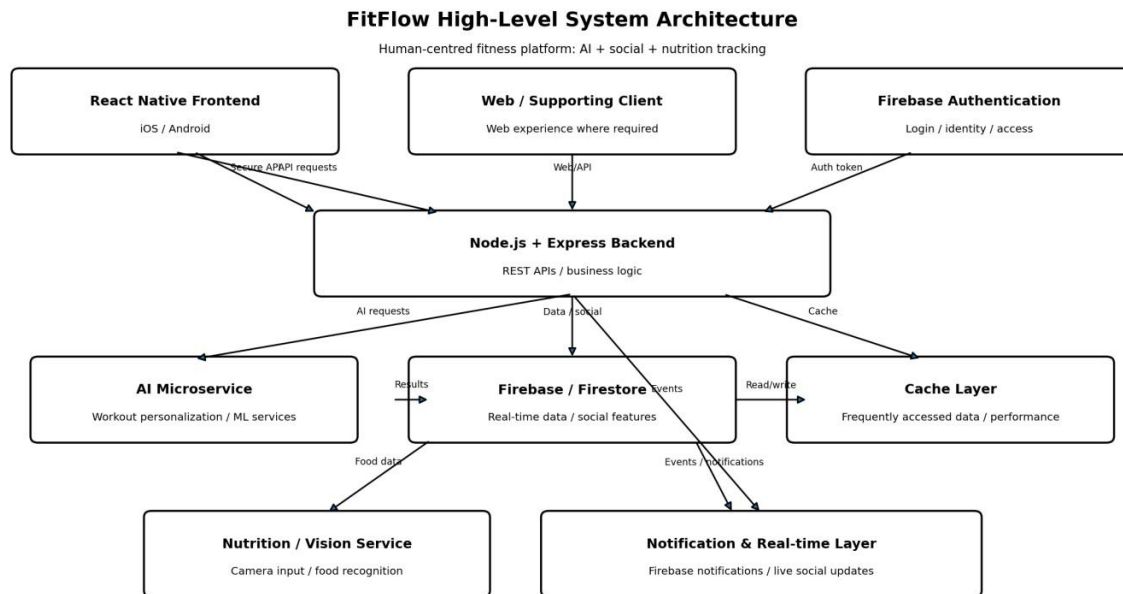
1.Objective

The objective is to design a high-level architecture for the FitFlow redesign using the technology stack selected through Activities 1–3. The architecture defines the frontend, backend services, AI microservice, database, caching and real-time layers, while showing the main data flows for personalized workouts, social sharing and nutrition tracking.

2.Selected Technology Stack

Layer	Technology / Component	Purpose
Frontend	React Native	Cross-platform mobile application for iOS and Android.
Backend	Node.js + Express	REST APIs and application/business logic.
AI layer	AI Microservice + ML services	Personalized workout recommendations and advanced AI processing.
Database / Real-time	Firebase / Firestore	Real-time social data and application data.
Authentication	Firebase Authentication	User identity and authentication.
Caching	Cache layer	Reduce repeated database access and improve response performance.
Notifications	Firebase notification / real-time layer	Scalable notifications and live social updates.

3.High-Level Architecture Diagram



4.Architecture Components

React Native Frontend: Provides the mobile user interface for workouts, social features, nutrition tracking and progress dashboards.

Node.js + Express Backend: Handles REST APIs, business logic, validation, integration with AI services and data services.

Firestore Authentication: Manages user authentication and provides identity information used for protected application resources.

AI Microservice: Processes AI-related requests such as personalized workout recommendations. Advanced AI models can be hosted through cloud services as described in the case study.

Firestore / Firestore: Supports application data and real-time social functionality, including social interactions and scalable notification-related services.

Cache Layer: Stores frequently accessed information temporarily to reduce repeated database operations and improve response time.

Nutrition / Vision Service: Receives camera-based food information and supports food recognition using computer-vision/ML capabilities.

Notification & Real-time Layer: Supports live social updates and scalable notifications required by the application.

5.Critical Data Flow 01 – Personalized Workout Plan

The user opens FitFlow through the React Native application and requests a personalized workout plan.

The frontend sends the request securely to the Node.js/Express backend.

The backend validates the user and gathers the relevant user/activity information.

The backend sends the AI-related request to the AI microservice.

The AI service generates or retrieves a personalized workout recommendation.

The recommendation is returned to the backend and then delivered to the mobile frontend.

The resulting plan can be stored for later access and displayed through the user's workout experience.

6.Critical Data Flow 02 – Social Sharing

The user creates or interacts with a social post, challenge or group activity in the React Native app.

The request is sent to the Node.js/Express backend.

The backend validates the request and user access.

The social data is written to Firebase/Firestore.

Real-time listeners update relevant users when social data changes.

The notification/real-time layer can deliver relevant notifications and live updates.

7.Critical Data Flow 03 – Nutrition Tracking

The user captures a food image using the mobile application camera.

The image/data is sent through the appropriate backend or nutrition/vision integration.

The computer-vision/ML service analyses the food information.

The recognized food information is returned to the backend.

The nutrition record is stored for the user and can be displayed in the nutrition/progress dashboard.

Only necessary information should be transmitted and retained, supporting the privacy-focused architecture described in the case study.

8.Security Considerations

Use Firebase Authentication for user identity and protected access.

Apply authorization rules so users can access only their own private information and permitted social resources.

Use secure communication between frontend, backend and external services.

Protect sensitive wellness and personal data through appropriate storage and access controls.

Minimize unnecessary data transmission, consistent with the case study's privacy-focused architecture.

Include accessibility and security checks as part of testing and deployment.

9.Scalability Considerations

Use stateless REST APIs where possible so backend instances can be scaled horizontally.

Use Firebase/Firestore real-time capabilities for growing social interactions and notifications.

Use caching to reduce repeated access to frequently requested information.

Keep AI processing separated as a microservice so AI workloads can be scaled independently.

Monitor application performance and usage after deployment and use analytics to guide continuous improvement.

10.Integration Considerations

React Native integrates with backend APIs and native device capabilities required for camera and notification features.

Node.js/Express acts as the main integration layer between the client, AI services and data services.

Firebase supports real-time social functionality and notifications.

The AI microservice can integrate with TensorFlow Lite/on-device capabilities and cloud AI services as required by the case study.

The architecture keeps major services separated so future features can be added without redesigning the entire system.

11. Architecture Decision Record (ADR)

ADR Title	Selection of the FitFlow High-Level Technology Architecture
Status	Accepted
Context	FitFlow requires a cross-platform mobile experience, high performance, AI-powered personalization, real-time social features, nutrition tracking, strong security and scalable notifications.
Decision	Use React Native for the frontend, Node.js + Express for backend services, Firebase/Firestore for real-time/application data, Firebase Authentication for identity, and a separate AI microservice for AI-related processing.
Rationale	This architecture supports rapid cross-platform development, real-time functionality, AI integration, maintainability and the privacy-focused approach described in the case study.
Consequences	The team must manage integration between the backend, Firebase and AI services. Native integrations may still be required for selected device capabilities, and AI/cloud usage must be monitored for cost and performance.

12. Conclusion

The proposed high-level architecture combines the selected FitFlow technologies into a modular structure.

React Native provides the cross-platform client, Node.js/Express provides the main application services, Firebase supports authentication and real-time functionality, and the AI microservice handles intelligent personalization and related AI processing. The architecture also includes caching, notifications and nutrition/vision integration to address the major requirements identified in the FitFlow redesign case study.